

Reinforcement Learning

Clase 06

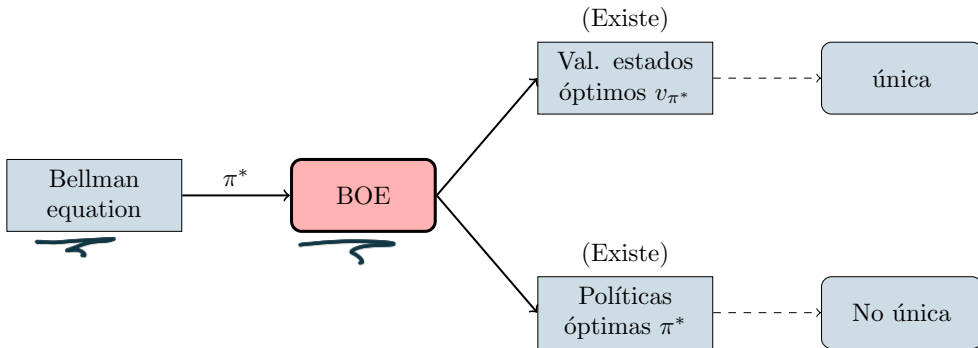
Ignacio Brevis

`ignaciobrve@unisabana.edu.co`

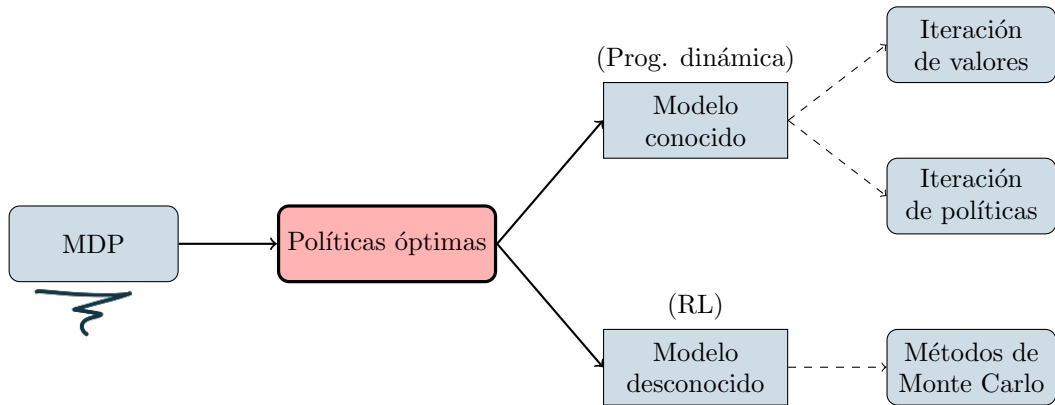
Universidad de La Sabana

23 septiembre 2026





- Policy iteration (Bellman equation) $\longleftarrow$ Generalized Policy Iteration (GPI)
- Value iteration (Bellman optimality equation)



- Usando modelo: Conocemos la distribución de probabilidad

$$\mathbb{E}[X] = \sum_x p(\underline{X = x})x$$

- Sin modelo: No conocemos la distribución de probabilidad. Podemos aproximar la esperanza con un muestreo u observaciones $\{x_1, x_2, \dots, x_n\}$ de X .

$$\mathbb{E}[X] \approx \bar{x} := \frac{1}{n} \sum_{j=1}^n x_j$$

(garantías por la ley de los grandes números) ✖

- Los **métodos de Monte Carlo no requieren conocer el modelo**. Esto es importante ya que:

$$\underline{v_\pi(s) = \mathbb{E}[G_t | S_t = s]}$$

$$\underline{q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]}$$

La idea fundamental en los métodos de MC para reinforcement learning es usar una estimación **model-free** de la función de valor de acción (q values), por medio de

$$q_{\pi_k} = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{n} \sum_{i=1}^n g_{\pi_k}^{(i)}(s, a)$$

Dada una política inicial π_0 , el algoritmo, en la k -ésima iteración:

❶ **Evaluación de la política:** Estimamos q_{π_k} para cada (s, a) . Específicamente, para cada par (s, a) obtenemos “suficientes” episodios para aproximar $q_{\pi_k}(s, a)$ por medio de $q_k(s, a)$.

❷ **Mejora de la política:** Actualizamos la política por medio de

$$\pi_{k+1}(s) \leftarrow \arg \max_{\pi} \pi(a|s) q_k(s, a),$$

para cada $s \in \mathcal{S}$. La política optimal (greedy) is $\pi_{k+1}(a_k^*|s) = 1$ con $a_k^* \in \arg \max_a q_k(a, s)$.

Algorithm 5.1: MC Basic (a model-free variant of policy iteration)

Initialization: Initial guess π_0 .

Goal: Search for an optimal policy.

For the k th iteration ($k = 0, 1, 2, \dots$), do

For every state $s \in \mathcal{S}$, do

For every action $a \in \mathcal{A}(s)$, do

Collect sufficiently many episodes starting from (s, a) by following π_k

Policy evaluation:

→ $q_{\pi_k}(s, a) \approx q_k(s, a) =$ the average return of all the episodes starting from (s, a)

Policy improvement:

$\left\{ \begin{array}{l} a_k^*(s) = \arg \max_a q_k(s, a) \\ \pi_{k+1}(a|s) = 1 \text{ if } a = a_k^*, \text{ and } \pi_{k+1}(a|s) = 0 \text{ otherwise} \end{array} \right.$

Algorithm 5.2: MC Exploring Starts (an efficient variant of MC Basic)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) .

Goal: Search for an optimal policy.

For each episode, do

→ **Episode generation:** Select a starting state-action pair (s_0, a_0) and ensure that all pairs can be possibly selected (this is the exploring-starts condition). Following the current policy, generate an episode of length T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

→ $\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

→ $\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

→ $q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

→ $\pi(a|s_t) = 1$ if $a = \arg \max_a q(s_t, a)$ and $\pi(a|s_t) = 0$ otherwise

- Episodios partiendo de cada (s, a) (Exploring-starts)
- Descomponer el episodio en subepisodios
- Actualizar la política mientras calculamos retornos

$$\pi(a|s) = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s), \end{cases} \quad \text{con} \quad a^*(s) \in \arg \max_a q^*(a, s) \quad [\text{política greedy}]$$

Soft policies

Llamaremos a una política soft si cada acción tiene una probabilidad positiva. Dado $s \in \mathcal{S}$,

$$\pi(a|s) > 0, \text{ para cada } a \in \mathcal{A}(s)$$

- Con políticas soft, unos pocos episodios suficientemente largos pueden cubrir todos los pares estado-acción.
- Elimina la necesidad de tener un gran número de episodios partiendo de cada par estado-acción. (requerido en *Exploring-starts*).

Soft policies: ε -greedy

En particular, nos interesaremos en la política ε -greedy que nos permite la **exploración y explotación**.

$$\pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{para la acción greedy} \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & \text{para las otras acciones} \end{cases}$$

Handwritten notes: g_k
 $g_1, g_2, \dots, g_k$
 $g_k \rightarrow g_{k+1}$

donde $\varepsilon \in (0, 1)$ y $|\mathcal{A}(s_t)|$ representa el número de acciones posibles en el estado s .

Handwritten: $\varepsilon = 0,2 \quad |\mathcal{A}(s)| = 5$

$$\pi(a|s) = \begin{cases} 0,84 \\ 0,04 \end{cases}$$
$$0,84 + 4 \cdot 0,04 = 1$$

Handwritten: $a = a^*$
 $a \neq a^*$

Handwritten: $\varepsilon = 0$ (greedy)

$$\pi(a|s) = \begin{cases} 1 & a = a^* \\ 0 & a \neq a^* \end{cases}$$

Handwritten: $\varepsilon = 1$

$$\pi(a|s) = \begin{cases} 0,2 & a = a^* \\ 0,2 & a \neq a^* \end{cases}$$

Tercer método: MC ε -greedy

Mejora de política: Elegimos a^* de la forma

$$a^* \leftarrow \arg \max_{\pi} q_{\pi_k}(s, a)$$

Pólítica greedy

$$\rightarrow \pi(a|s) = \begin{cases} 1, & a = a^*, \\ 0, & a \neq a^*, \end{cases}$$

(full explotación)

Pólítica ε -greedy

$$\rightarrow \pi(a|s) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & a = a^*, \\ \frac{\varepsilon}{|\mathcal{A}(s)|}, & a \neq a^*, \end{cases}$$

(exploración y explotación)

Tercer método: MC ϵ -greedy

Algorithm 5.3: MC ϵ -Greedy (a variant of MC Exploring Starts)

Initialization: Initial policy $\pi_0(a|s)$ and initial value $q(s, a)$ for all (s, a) . $\text{Returns}(s, a) = 0$ and $\text{Num}(s, a) = 0$ for all (s, a) . $\epsilon \in (0, 1]$

Goal: Search for an optimal policy.

For each episode, do

Episode generation: Select a starting state-action pair (s_0, a_0) (the exploring starts condition is not required). Following the current policy, generate an episode of length

T : $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$.

Initialization for each episode: $g \leftarrow 0$

For each step of the episode, $t = T - 1, T - 2, \dots, 0$, do

$g \leftarrow \gamma g + r_{t+1}$

$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$

$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$

Policy evaluation:

$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$

Policy improvement:

Let $a^* = \arg \max_a q(s_t, a)$ and

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|}\epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|}\epsilon, & a \neq a^* \end{cases}$$

Stochastic approximation: Example

Ejemplo: Dada una variable aleatoria X , queremos estimar $\mathbb{E}[X]$. Sean $\{x_1, x_2, \dots, x_n\}$ realizaciones de X . Sabemos que

$$\mathbb{E}[X] \approx \underline{w_{k+1}} = \frac{1}{k} \sum_{i=1}^k x_i$$

Notemos que $\underline{w_{k+1}} = \underline{w_k} - \frac{1}{k}(w_k - \underline{x_k})$

$$w_{k+1} = \frac{1}{k} (x_1 + \dots + x_{k-1} + x_k)$$

$$w_k = \frac{1}{k-1} (x_1 + \dots + x_{k-1})$$

$$w_3 = \frac{1}{2} (x_1 + x_2)$$

$$w_4 = \frac{1}{3} (\underline{x_1 + x_2} - x_3)$$

$$w_4 = w_3 - \frac{1}{3} (w_3 - x_3)$$

Stochastic approximation: Example

Notemos que w_{k+1} = $w_k - \frac{1}{k}(w_k - x_k)$

$$w_k - \frac{1}{k}(w_k - x_k)$$

$$\frac{1}{k-1}(x_1 + \dots + x_{k-1}) - \frac{1}{k} \left[\frac{1}{k-1}(x_1 + \dots + x_{k-1}) - x_k \right]$$

$$\frac{1}{k-1}(\dots) - \frac{1}{k} \frac{1}{k-1}(\dots) + \frac{1}{k} x_k$$

$$\frac{1}{k-1}(\dots) \left[1 - \frac{1}{k} \right] + \frac{1}{k} x_k$$

$$\frac{1}{\cancel{k-1}} \left(\frac{\cancel{k-1}}{k} \right) (x_1 + \dots + x_{k-1}) + \frac{1}{k} x_k$$

$$\frac{1}{k} (x_1 + \dots + x_{k-1}) + \frac{1}{k} x_k = \frac{1}{k} (x_1 + \dots + x_{k-1} + x_k) = w_{k+1}$$

$$w_{k+1} = \frac{1}{k} (x_1 + \dots + x_{k-1} + x_k)$$

$$w_k = \frac{1}{k-1} (x_1 + \dots + x_{k-1})$$

Aproximación estocástica se refiere a una amplia clase de algoritmos iterativos para resolver problemas de raíces y problemas de optimización.

El algoritmo de Robbins-Monro (1951), permite resolver problemas de la forma

$$g(w) = 0$$

cuando **no conocemos g y solo tenemos acceso a observaciones con ruido** $\tilde{g}(w, \eta) := g(w) + \eta$ por medio de

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k)$$

H. Robbins and S. Monro, A Stochastic Approximation Method, *Ann. Math. Stat. or Ann. Math. Statist.*, 1951.

A STOCHASTIC APPROXIMATION METHOD¹

By HERBERT ROBBINS AND SUTTON MONRO

University of North Carolina

1. Summary. Let $M(x)$ denote the expected value at level x of the response to a certain experiment. $M(x)$ is assumed to be a monotone function of x but is unknown to the experimenter, and it is desired to find the solution $x = \theta$ of the equation $M(x) = \alpha$, where α is a given constant. We give a method for making successive experiments at levels $x_1, x_2, \dots$ in such a way that x_n will tend to θ in probability.

2. Introduction. Let $M(x)$ be a given function and α a given constant such that the equation

$$(1) \quad M(x) = \alpha$$

has a unique root $x = \theta$. There are many methods for determining the value of θ by successive approximation. With any such method we begin by choosing one or more values $x_1, \dots, x_r$ more or less arbitrarily, and then successively obtain new values x_n as certain functions of the previously obtained $x_1, \dots, x_{n-1}$, the values $M(x_1), \dots, M(x_{n-1})$, and possibly those of the derivatives $M'(x_1), \dots, M'(x_{n-1})$, etc. If

$$(2) \quad \lim_{n \rightarrow \infty} x_n = \theta,$$

Stochastic approximation: Robbins-Monro algorithm

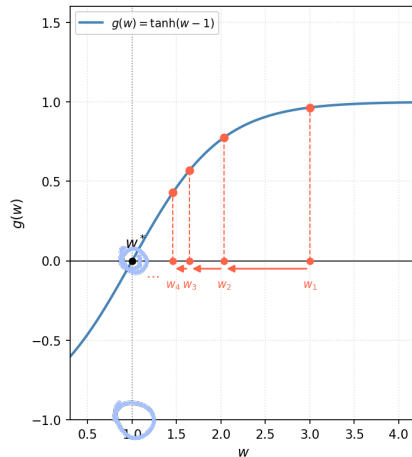
$$g(w) = \tanh(w - 1) = 0$$

Ejercicio: Generar un código para encontrar la solución de $g(w) = \tanh(w - 1) = 0$. Usando

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k)$$

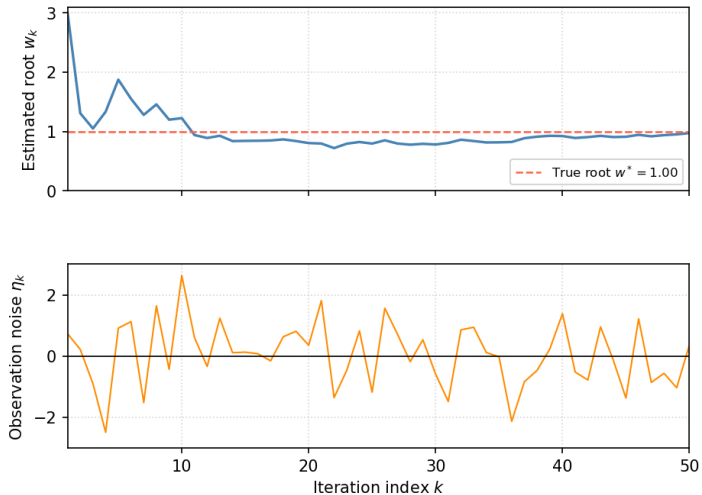
$\tanh(w - 1) + \text{ruido}$

- $w_1 = 3.$
- $\alpha_k = \frac{1}{k}$
- $\tilde{g}(w, \eta) = g(w) + \eta$
- $\eta_k \sim \mathcal{N}(0, 1)$



Stochastic approximation: Robbins-Monro algorithm

Illustrative example of the RM algorithm
 $g(w) = \tanh(w - 1)$, $w_1 = 3.0$, $a_k = 1/k$, $\eta_k \sim \mathcal{N}(0, 1)$



Stochastic approximation: Robbins-Monro algorithm

Volviendo a la aplicación del valor esperado. Definamos

- $g(w) = w - \mathbb{E}[X]$
- $\tilde{g}(w, \eta) = w - x$ (con x muestra *iid.* de X)

entonces, queremos encontrar w^* tal que $g(w^*) = 0$

$$g(w) = w - \mathbb{E}(X)$$

$$\tilde{g}(w, \eta) = w - x$$

$$\tilde{g}(w, \eta) = \underbrace{w - \mathbb{E}(X)}_{g(w)} + \underbrace{\mathbb{E}(X) - x}_{\eta \text{ ruido}}$$

$$\begin{aligned} w_{k+1} &= w_k - \alpha_k \tilde{g}(w_k, \eta_k) \\ &= w_k - \alpha_k (w_k - x_k) \\ &= w_k - \frac{1}{k} (w_k - x_k) \end{aligned}$$

$$w_{k+1} = w_k - \frac{1}{k} (w_k - x_k)$$

Stochastic approximation: Robbins-Monro algorithm

Ejercicio: valor esperado de una función v . tal que

$$w = \mathbb{E}[v(X)]$$

con muestras x iid. de X

- $g(w)$?
- $\tilde{g}(w, \eta)$?
- η ?
- Algoritmo de Robbins-Monro ?

$$g(w) = w - \mathbb{E}(v(X))$$

$$\tilde{g}(w, \eta) = w - v(x)$$

$$= \underbrace{w - \mathbb{E}(v(X))}_{g(w)} + \underbrace{\mathbb{E}(v(X)) - v(x)}_{\text{ruido}}$$

$$w_{k+1} = w_k - \alpha_k (w_k - v(x_k))$$

Stochastic approximation: Robbins-Monro algorithm

La convergencia del algoritmo de Robbins-Monro viene dada por los siguientes teoremas:

Theorem (Robbins-Monro theorem)

Si en el algoritmo Robbins-Monro se cumple que:

❶ $0 < c_1 \leq \nabla_w g(w) \leq c_2$ para todo w ;

❷ $\sum_{k=1}^{\infty} a_k = \infty$ y $\sum_{k=1}^{\infty} a_k^2 < \infty$;

❸ $\mathbb{E}[\eta_k | \mathcal{H}_k] = 0$ y $\mathbb{E}[\eta_k^2 | \mathcal{H}_k] < \infty$;

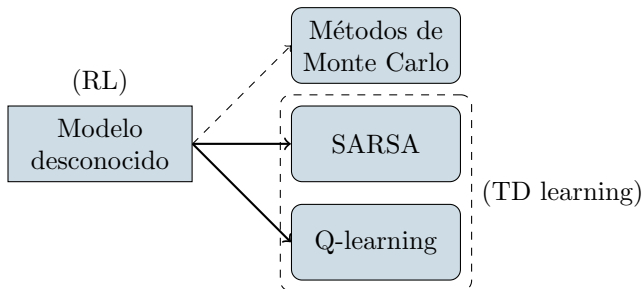
donde $\mathcal{H}_k = \{w_k, w_{k-1}, \dots\}$, entonces w_k *converge casi seguramente* a la raíz w^* tal que $g(w^*) = 0$.

Obs: ver también teorema de Dvoretzky

Temporal-difference (TD) methods

Los métodos de **temporal-difference** (TD) learning combinan ideas de los métodos de Monte Carlo y de programación dinámica (DP).

- **MC: No requiere modelos** y aprende de las experiencias (requiere completar un episodio antes de calcular retornos)
- **DP:** Requiere modelo y utiliza **bootstraps** (actualiza $v(s)$ usando $v(s')$ de los estados vecinos)



Learning to Predict by the Methods of Temporal Differences

RICHARD S. SUTTON

(RICH@GTE.COM)

GTE Laboratories Incorporated, 40 Sylvan Road, Waltham, MA 02254, U.S.A.

(Received: April 22, 1987)

(Revised: February 4, 1988)

Keywords: Incremental learning, prediction, connectionism, credit assignment, evaluation functions

Abstract. This article introduces a class of incremental learning procedures specialized for prediction - that is, for using past experience with an incompletely known system to predict its future behavior. Whereas conventional prediction-learning methods assign credit by means of the difference between predicted and actual outcomes, the new methods assign credit by means of the difference between *temporally successive predictions*. Although such *temporal-difference methods* have been used in Samuel's checker player, Holland's bucket brigade, and the author's Adaptive Heuristic Critic, they have remained poorly understood. Here we prove their convergence and optimality for special cases and relate them to supervised-learning methods. For most real-world prediction problems, temporal-difference methods require less memory and less peak computation than conventional methods *and* they produce more accurate predictions. We argue that most problems to which supervised learning is currently applied are really prediction problems of the sort to which temporal-difference methods can be applied to advantage.

The earliest and best-known use of a TD method was in Samuel's (1959) celebrated checker-playing program. For each pair of successive game positions, the program used the difference between the evaluations assigned to the two positions to modify the earlier one's evaluation. Similar methods have been used in Holland's (1986) bucket brigade, in the author's Adaptive Heuristic Critic (Sutton, 1984; Barto, Sutton & Anderson, 1983), and in learning systems studied by Witten (1977), Booker (1982), and Hampson (1983). TD methods have also been proposed as models of classical conditioning (Sutton & Barto, 1981a, 1987; Gelperin, Hopfield & Tank, 1985; Moore et al., 1986; Klopf, 1987).

Nevertheless, TD methods have remained poorly understood. Although they have performed well, there has been no theoretical understanding of how or why they worked. One reason is that they were never studied independently,

R. S. Sutton, Learning to predict by the methods of temporal differences, *Mach. Learn.*, 1988.

Temporal-difference (TD) methods

Bellman equation

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s],$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[G_{t+1} | S_t = s] \quad [\text{linealidad}]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \sum_{s'} \mathbb{E}[G_{t+1} | S_t = s, S_{t+1} = s'] p(s' | s) \quad [\text{esp. totales}]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \sum_{s'} \mathbb{E}[G_{t+1} | S_{t+1} = s'] p(s' | s) \quad [\text{prop. Markov}]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \sum_{s'} v_{\pi}(S_{t+1}) p(s' | s) \quad [\text{def. función valor}]$$

$$= \mathbb{E}[R_{t+1} | S_t = s] + \gamma \mathbb{E}[v_{\pi}(S_{t+1}) | S_t = s] \quad [\text{def. espe. cond.}]$$

$$= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] \quad [\text{linealidad}]$$

$$G_{t+1} \neq v_{\pi}(S_{t+1})$$
$$\mathbb{E}(G_{t+1} | S_t = s) = \mathbb{E}(v_{\pi}(S_{t+1}) | S_t = s)$$

TD methods: Learning states values

Bellman equation

$$\star \quad v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s], \quad s \in \mathcal{S}$$

Bellman expectation equation

$$\star \quad \underline{v_{\pi}}(s) = \mathbb{E}[R_{t+1} + \gamma \underline{v_{\pi}}(S_{t+1}) | S_t = s], \quad s \in \mathcal{S}$$

$$\underline{v_{\pi}(s) - \mathbb{E}(\cdot)} = 0$$

Robbins-Monro para Bellman expectation equation

$$\longrightarrow \underline{g(v(s))} := \underline{v(s) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]}$$

$$\underline{\tilde{g}(v(s))} := \underline{v(s) - (r_{t+1} + \gamma v_{\pi}(s_{t+1}))}$$

$$v(s) - \mathbb{E}(v(X))$$

RM
 $\longrightarrow$

$$\underline{v_{k+1}}(s_t) := \underline{v_k}(s_t) - \alpha_k \left[\underline{v_k}(s_t) - (r_{t+1} + \gamma \underline{v_{\pi}}(s_{t+1})) \right]$$

$\nearrow v_k$

TD methods: Learning states values

Dada una política π , el objetivo es calcular $v_\pi(s)$ para todo $s \in \mathcal{S}$. Supongamos que tenemos muestras de algunas experiencias generadas por π :

$$(s_0, r_1, s_1, r_2, s_2, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$$

Entonces, queremos estimar la **función de valor** por medio del algoritmo de temporal-difference:

$$\left\{ \begin{array}{l} v_{t+1}(s_t) = v_t(s_t) - \alpha(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) \right], \\ v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t \end{array} \right. \quad (1)$$

$$(2)$$

Obs: El algoritmo de temporal-difference (como modificación de Robbins-Monro) converge a $v_\pi(s_t)$.

Obs: El término “temporal-difference learning” normalmente se refiere a una clase más amplia de algoritmos de RL.



- Gymnasium es una biblioteca de Python que ofrece entornos de RL.
- Continuación de OpenAI Gym.
- Mantenida desde 2022 por la Farama Foundation (<https://farama.org>).
- Usaremos una version de gymnasium ≥ 1.3 .
- Pueden ver <https://github.com/ibreviis/in-deep-week-intro-RL> si necesitan ayuda con la instalación.

<https://gymnasium.farama.org/index.html>